

Nexus Logistics — Presenter Script

Avishkara'26 · JP-019 · Team Code with Errors

Naga Shreeshyl K S · Nivetha · Ankitha · Skanda

How to use this script

- Open /presentation for slides 1–6, then the **Optimizer Lab** (/optimizer).
- Speak naturally — these lines are **prompts**, not something to read word-for-word.
- Never invent ₹ ROI or production claims. If a metric is on screen, read **that** number.
- Demo data is **synthetic Bengaluru** (allowed by JP-019). Say that once early and once before the Lab.

Suggested flow: Slides 1 → 5 → Live Lab (Overview → Scenario → Optimize → Plan → Risk → Analytics → Exceptions optional → Evidence → Exports) → Slide 6 Thank you.

Part A — Presentation slides

Slide 1 — Team · JP-019

On screen: NEXUS LOGISTICS · team cards · Avishkara'26 · JP-019

Say:

Good morning / afternoon. We are **Code with Errors**.
Our project is **Nexus Logistics** for problem statement **JP-019** — Last-Mile Delivery Route Optimizer and Late-Delivery Risk Predictor.
On the team: **Naga Shreeshyl K S** — Team Leader and Developer; **Nivetha** and **Ankitha** — Research and Presenter; **Skanda** — Tester.
In the next few minutes we'll show the problem, how Nexus solves it, what we built with, and then a **live Lab demo**.

Do: Hover or tap a teammate card once if you want energy; then Next.

Slide 2 — The problem

On screen: Pressure-point cards · Bengaluru same-day example

Say:

Last-mile delivery is hard for a reason.

We have **many timed orders, limited vans, capacity limits**, and **customer time windows**. We also need to know which stops may still arrive late.

Tap with me: limited vans... tight windows... critical orders... late risk.

Real example — **Bengaluru same-day**: twenty-plus stops across Koramangala, Indiranagar, JP Nagar; only four to six vans; some medicine or same-day criticals; windows like eleven to one.

A simple “nearest next stop” plan looks short on the map — then vans overfill, windows break, and late deliveries pile up.

The goal is not the shortest line. The goal is a plan vans can actually run today.

Do: Click each pain card once while speaking.

Slide 3 — Our solution

On screen: Three steps — Build routes · Warn late risk · Prove it

Say:

Nexus does three things.

One — Build routes. Google OR-Tools assigns stops to vans under **hard** capacity and time windows.

Two — Warn late risk. A classical ML model scores which stops may still go late. It **warns** — it never breaks the rules.

Three — Prove it. On the same day we compare a naive **Baseline** against **Nexus** and show measured late count, distance, and on-time percent.

If the day cannot fit, we return a **partial plan** and list deferred stops. We do **not** fake full success.

Do: Hover the three step cards as you name them.

Slide 4 — Tech stack

On screen: Flip / reorder tech cards

Say:

Quick stack — in plain English.

React and Vite for the Lab and this deck. **Leaflet** for the Bengaluru map — visualization only; JP-019 does not require GPS hardware.

FastAPI is the API behind Optimize. **Google OR-Tools** is the real route planner. **scikit-learn** scores late risk. **SQLite** holds demo scenarios. Travel is **OSRM when available, otherwise Haversine**, and we always show which one ran. Important for judges: **we do not use ChatGPT or any LLM to invent routes**. Hard constraints stay hard.

Do: Flip 2–3 cards (OR-Tools, scikit-learn, No LLM). Optional: drag one card to show interactivity. Don't spend more than ~60–90 seconds here.

Slide 5 — See the Lab

On screen: Demo path tiles · Open Lab button

Say:

Now we open the application.
The Lab uses **generated synthetic Bengaluru data** — allowed by JP-019. The **solver and risk scores run for real** on that data.
Path we'll follow: Overview → Scenario → Optimize → Plan side-by-side → Risk → Analytics → Evidence → Exports — then back here for Thank you.
Let's go.

Do: Click **Open Lab · Overview**. (Optional: tap tiles to mark the path.)

Slide 6 — Thank you

On screen: Thank you · live metrics if you ran Optimize · honest limits

Say (after returning from Lab):

Thank you.
If the numbers from our demo are on screen — late and kilometres Baseline versus Nexus — those came from **this session**, not from a slide graphic.
Honest limits: synthetic demo data; ML not production-validated; the optimizer is **time-limited** — feasible does not mean globally optimal; travel may fall back to Haversine.
We're happy to reopen any Lab page or take questions.
Team **Code with Errors** — Nexus Logistics — Avishkara twenty-six — JP-019. Thank you.

Do: Expand “Honest limits” if judges ask. Offer Back to Lab if they want a replay.

Part B — Optimizer Lab pages

Use the **sticky top bar**: **Load** · **Optimize** · **Play** · **speed** · **Rush** · **Win**.

Lab — Overview (/optimizer)

What it is: Big-picture home for the demo session.

Say:

This is the **Optimizer Lab Overview**.

From here we load a synthetic day, run Optimize, and jump into Plan, Risk, and Analytics. Everything you see after Optimize is computed live for this session.

Do: Stay brief. Click **Load** if no scenario yet, or go to Scenario.

Lab — Scenario (/optimizer/scenario)

What it is: Choose / inspect the day (orders, vans, archetype).

Say:

On **Scenario** we set up the day — synthetic Bengaluru orders, fleet, and pressure archetype if we use one — balanced, surge, tight windows, or fleet short.

I'll **Load** a fresh synthetic day so judges see the data appear.

Do: Press **Load** (or pick archetype then Load). Point at order / vehicle counts. Then say “Now Optimize.”

Lab — Optimize (top bar action)

Not a separate page — runs from the sticky bar; then usually lands on **Plan**.

Say:

Pressing **Optimize** runs Baseline and Nexus on the **same** scenario.

Baseline is the naive plan. Nexus is our constrained optimizer.

When it finishes, we'll compare them side by side.

Do: Press **Optimize**. Wait for the overlay. Don't talk over a long solve — fill with: "Hard capacity and time windows stay hard while the solver searches."

Lab — Plan (/optimizer/plan)

What it is: Side-by-side Baseline vs Nexus maps · measured improvement · play day.

Say:

This is **Plan**. Left is **Baseline**, right is **Nexus**, same day.
Look at the measured improvement — late deliveries and distance before and after.
I'll press **Play** so vans move through the day. Speed control is here if we need to go faster.
Notice travel source on the solution — roads or Haversine — so you know how distance was built.

Do: Point at both maps. Press Play once. Call out one clear win (e.g. late 9 → 0) **only if that is on screen**.

Lab — Risk (/optimizer/risk)

What it is: Late-delivery risk overlay / high-risk stops.

Say:

Risk shows the ML late-delivery scores.
High-risk stops are highlighted for operators.
This score is **advisory**. It does **not** relax capacity or time windows. The optimizer still owns feasibility.

Do: Select a high-risk stop if one is visible. Keep this under 30–45 seconds.

Lab — Exceptions (/optimizer/exceptions)

What it is: Disruptions / rush / before–after when the day breaks.

Say:

Exceptions is where the day breaks — van unavailable, rush order, replan.
JP-019 cares about disruption handling. I'll inject a **Rush** order from the top bar, then re-

optimize if needed, and show how the plan adapts.
We show honest before and after — not a fake perfect day.

Do: Optional but strong for judges. Use **Rush** once. If short on time, skip and say “we can return to Exceptions in Q&A.”

Lab — Analytics (/optimizer/analytics)

What it is: Live Baseline vs Nexus bars · on-time % · history.

Say:

Analytics graphs the **same Optimize results** — not hardcoded slide art.
Each row is scaled to its own max, so nine versus zero actually looks like nine versus zero.
Grey is Baseline, coral is Nexus. On-time percent is derived from late count and order count.
History below updates as we run more solves.

Do: Point at Late and Distance rows specifically. Don't claim ROI.

Lab — Evidence (/optimizer/evidence)

What it is: Checklist / proof panel for JP-019 claims.

Say:

Evidence is our honesty board — what JP-019 asked for and what this run actually proved:
constrained routing, risk, compare, partial or deferred when needed, synthetic data disclosed.
Judges can scroll this if they want receipts without trusting a pitch line.

Do: Tick through 2–3 checklist items verbally.

Lab — Exports (/optimizer/exports)

What it is: Driver sheet / map export / weather · return to Thank you.

Say:

Exports closes the operator loop — driver sheet, map GeoJSON, weather refresh for Bengaluru.
Demo complete. I'll return to the presentation **Thank you** slide.

Do: Click **Back to presentation · Thank you**. Optional: flash one export button without downloading if time is tight.

Part C — One-minute emergency pitch

If you only have ~60 seconds:

We're Code with Errors — Nexus for JP-019.
Last-mile needs feasible vans under capacity and windows, plus late risk warnings.
We use OR-Tools, classical ML, and a live Lab on synthetic Bengaluru data.
Optimize shows Baseline versus Nexus with measured late and kilometres.
No LLM inventing routes. Thank you.

Part D — Likely judge questions (short answers)

Question	Answer
Is the data real?	Synthetic Bengaluru scenarios for the hackathon — disclosed. Solver/ML run for real on that data.
Why not ChatGPT for routes?	Routing is mathematical optimization with hard constraints. LLMs don't guarantee capacity or windows.
Is the solution optimal?	Time-limited search — we claim feasible under the limit , not globally optimal.
Does ML change the route rules?	No. Advisory only. Never relaxes hard constraints.
What about GPS / live maps?	Map is visualization (Leaflet). Travel via OSRM or Haversine fallback; we show <code>travel_source</code> .
Partial plans?	

Question	Answer
	If the day can't fit, we return assigned routes plus deferred stops — no fake 100% success.

Demo checklist (print / keep beside laptop)

1. ☐ Open /presentation fullscreen
2. ☐ Slides 1–5 (~3–4 min)
3. ☐ Lab Overview → Load → Optimize
4. ☐ Plan — Play once — call out live metrics
5. ☐ Risk — 20 seconds
6. ☐ Analytics — point at true bar lengths
7. ☐ Exceptions — Rush if time
8. ☐ Evidence — 20 seconds
9. ☐ Exports → Thank you (slide 6)
10. ☐ Q&A ready

Team: Code with Errors · **Product:** Nexus Logistics · **Event:** Avishkara'26 · **PS:** JP-019